PIPELINE EXPERIMENTAL LLM-ADA-PROJECT

Zero-shot, 3-shot, 5-shot e fine-tuning (LoRA) em quatro LLMs médicos

Documentação técnica do repositório – outubro de 2026

Disponível em https://github.com/perrijuan/llm-ada-project

Resumo

O repositório compara quatro LLMs abertos (Gemma 4 E4B, HuatuoGPT-o1-7B, MedGemma 1.5 4B e Qwen3.5-4B) em quatro condições: **zero-shot**, **3-shot**, **5-shot** (demonstrações no prompt, sem treino) e **fine-tuned** (SFT com adaptadores LoRA sobre o checkpoint em 4 bits). A avaliação usa as mesmas 48 perguntas MedQuAD, 100 questões MedQA e 100 PubMedQA, além de um benchmark de velocidade (TTFT, TTLT, tokens/s). `rodar_todos.py` executa um processo isolado por execução, `registro.py` grava logs e tabelas, e `consolidar.py` e `resumir.py` produzem as tabelas finais. Este documento descreve o código da versão atual (protocolo `icl_0_3_5_ft5_personas_v4_ft_separado`).

SUMÁRIO

1 Visão geral do pipeline **2**

2 Modelos e condições experimentais **3**

2.1 Os quatro modelos 3

2.2 As quatro condições 3

3 Protocolo experimental **4**

3.1 Dados e splits 4

3.2 Seeds 4

3.3 Personas e estrutura das mensagens 5

3.4 Fine-tuning (SFT com LoRA) 5

3.5 Guardrails 6

4 Orquestração com rodar_todos.py **6**

4.1 Opções 7

4.2 Ordem e isolamento 7

4.3 Estados, falhas e retomada 7

5 Artefatos e logs **8**

5.1 Estrutura de uma campanha 8

5.2 registro.py 8

5.3 Pasta auditoria/ 9

6 Consolidação e métricas **10**

6.1 Métricas por execução 10

6.2 consolidar.py 10

6.3 resumir.py 10

7 Como reproduzir **11**

8 Limitações e pontos de atenção **12**

1 VISÃO GERAL DO PIPELINE

O experimento tem três camadas. O **executor** (`rodar_todos.py`) decide a ordem das execuções e abre um processo Python novo para cada uma. O **script do modelo** (`modelos/<modelo>.py`) prepara os dados, avalia as quatro condições e grava resultados. As **ferramentas de pós-processamento** (`consolidar.py` e `resumir.py`) juntam as execuções concluídas e calculam os resumos estatísticos.

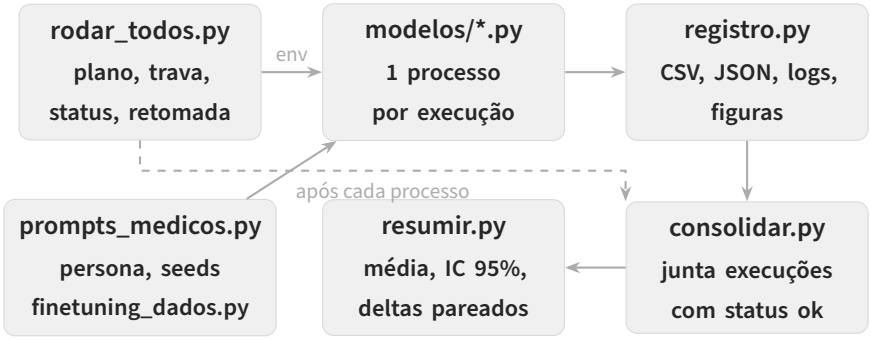


FIGURA 1: Fluxo entre os arquivos principais. A consolidação é chamada pelo executor após cada processo e pode ser repetida manualmente.

Dentro de cada processo, o script do modelo segue sempre a mesma sequência:

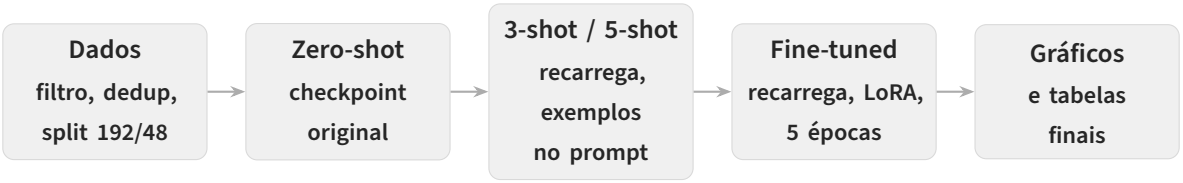


FIGURA 2: Sequência em um script de modelo. Em cada condição: MedQuAD (respostas, F1, ROUGE-L, NLL), velocidade, MedQA e PubMedQA.

A ordem das condições é apenas operacional. Cada condição recarrega o checkpoint original com `build_model()`, então nenhuma condição herda pesos ou estado da anterior.

2 MODELOS E CONDIÇÕES EXPERIMENTAIS

2.1 OS QUATRO MODELOS

Os quatro scripts em `modelos/` são praticamente idênticos: mudam o nome curto, o checkpoint, a pasta de saída e, no HuatuoGPT, o tratamento da resposta de raciocínio.

Chave	MODEL_ID	MODEL_NAME	Particularidade
gemma4	unsloth/gemma-4-E4B-it	Gemma 4 E4B	Script de referência.
huatuogpt	FreedomIntelligence/HuatuoGPT-o1-7B	HuatuoGPT-o1-7B	Extrai o texto após `## Final Response`; previews com até 1.024 tokens.
medgemma	unsloth/medgemma-1.5-4b-it	MedGemma 1.5 4B	Igual ao Gemma, salvo o checkpoint.
qwen35	unsloth/Qwen3.5-4B	Qwen3.5-4B	Raciocínio desligado no chat template.

TABELA 1: Modelos avaliados, na ordem padrão de execução.

Todos carregam com `FastModel.from_pretrained` (Unsloth), em 4 bits (`LOAD_IN_4BIT=True`) e com contexto de inferência `MODEL_CONTEXT_LENGTH=16384`. O chat template é sempre o do próprio tokenizer; `enable_thinking=False` só tem efeito nos templates que aceitam essa opção.

2.2 AS QUATRO CONDIÇÕES

As condições são definidas por uma única lista, lida como *(nome, número de demonstrações, treina?)*:

```
CONDITIONS = [('zero-shot', 0, False), ('3-shot', 3, False),
              ('5-shot', 5, False), ('fine-tuned', 0, True)]
```

Condição	Demonstrações	Treina pesos	O que muda
zero-shot	0	não	Apenas `system` (persona + instrução) e a pergunta.
3-shot	3	não	Três pares pergunta/resposta antes da pergunta de teste.
5-shot	5	não	Os mesmos três do 3-shot, mais dois, na mesma ordem.
fine-tuned	0	sim (LoRA)	SFT em 192 pares MedQuAD; avaliado sem demonstrações.

TABELA 2: Condições experimentais. “Shots” contam demonstrações resolvidas, não personas.

Few-shot (ICL) e fine-tuning são fluxos separados: as demonstrações nunca entram no `SFTTrainer`, e o formatador de SFT (`format_training_example`) não chama a seleção de demonstrações.

3 PROTOCOLO EXPERIMENTAL

3.1 DADOS E SPLITS

- **MedQuAD** (`keivalya/MedQuad-MedicalQnADataset`): filtro `keep_example` (pergunta ≥ 12 caracteres, resposta entre 40 e 900 caracteres, sem prefixos genéricos como “For more information…”), deduplicação por pergunta normalizada (espaços e caixa), sorteio de 240 registros e split **192 treino / 48 teste** com `split_seed`.
- **Banco de demonstrações MedQuAD**: `indices_demonstracoes` (em `finetuning_dados.py`) só aceita perguntas filtradas que não estão nem no treino nem no teste; exige pelo menos 5.
- **MedQA** (`GBaker/MedQA-USMLE-4-options`): 100 questões do split `test`; demonstrações do split `train`, sem perguntas do teste.
- **PubMedQA** (`qiaojin/PubMedQA`, `pqa_labeled`): 100 questões para teste; demonstrações dos registros restantes. Respostas *yes/no/maybe* viram A/B/C.
- **Velocidade**: as 20 primeiras perguntas do teste MedQuAD (`SPEED_N=20`), com exatamente 160 tokens gerados.
- Os conjuntos usados ficam em `registros/dados_*.csv`, o que permite auditar a separação treino/teste/demonstrações.

3.2 SEEDS

Há dois tipos de seed, separados de propósito:

Seed	Opção	Controla
`split_seed`	`--split-seed 42`	Subconjunto de 240, split 192/48, perguntas MedQA/PubMedQA. Fica fixa.
`seed`	`--seeds 42 123 2026`	Seleção das demonstrações, `set_seed` antes de cada condição, `random_state` do LoRA e seed do `SFTConfig`.

TABELA 3: Seeds. Elas chegam ao script pelas variáveis `LLM_SPLIT_SEED` e `LLM_SEED`.

A seleção de demonstrações é determinística por pergunta: um gerador é criado a partir de `sha256(seed|tarefa|pergunta)`, sorteia 5 exemplos do banco, e o 3-shot usa os 3 primeiros. Assim, a mesma pergunta recebe os mesmos exemplos em todas as condições e modelos com a mesma seed; o 5-shot difere do 3-shot apenas pelos dois exemplos acrescentados.

```
digest = hashlib.sha256(f'{SEED}|{task}|{question}'.encode('utf-8')).
   digest()
rng = random.Random(int.from_bytes(digest[:8], 'big'))
return rng.sample(demo_pools[task], 5)[:shots]
```

A variação entre seeds mistura ICL e aleatoriedade do treino, mas não mede variação do split. Repetições com a mesma seed (`--repeticoes`) medem repetibilidade, não amostras independentes, e não há garantia de determinismo numérico em todas as GPUs.

3.3 PERSONAS E ESTRUTURA DAS MENSAGENS

`prompts_medicos.py` define três personas (textos completos em `prompts_prontos.json`; fontes em `PROMPTS_E_PROTOCOLO.md`):

- `clinico` (padrão): médico generalista, adaptado do MedAgents.
- `biomedico`: pesquisador biomédico, adaptado da model card OpenBioLLM.
- `original`: instruções do pacote recebido, sem persona.

O prompt de sistema é *prefixo da persona + instrução da tarefa*: "Answer the medical question directly in 2-4 factual sentences." no MedQuAD e "Return only the option letter…" em MedQA/PubMedQA. A persona é a mesma nas quatro condições de uma campanha; trocar a persona exige outra pasta de saída.

```
system:    persona + instrucao da tarefa
user:      pergunta de demonstracao 1
assistant: resposta de referencia 1
...        (3 ou 5 pares; nenhum no zero-shot e no fine-tuned)
user:      pergunta de teste (sem resposta)
```

3.4 FINE-TUNING (SFT COM LORA)

O braço *fine-tuned* recarrega o checkpoint base, aplica `FastModel.get_peft_model` e treina com `SFTTrainer` (TRL). Cada exemplo de treino é uma conversa *system + pergunta + resposta de referência*.

Parâmetro	Valor	Parâmetro	Valor
Exemplos de treino	192	Épocas	5 (`max_steps=-1`)
Quantização base	4 bits (QLoRA)	Learning rate	2×10^{-5}, linear
LoRA r / α / dropout	8 / 16 / 0	Warm-up	5 passos
Módulos	atenção + MLP (linguagem)	Otimizador	`adamw_torch`, wd 0,01
Microbatch × acumulação	$2 \times 16 = 32$	Comprimento SFT	768 tokens
Precisão	bf16 se suportado, senão fp16	Checkpointing	`'unsloth'`

TABELA 4: Hiperparâmetros de `train_regime()`. O adapter e o tokenizer são salvos em `<OUTPUT_DIR>/adapter-fine-tuned`.

3.5 GUARDRAILS

O código prefere falhar com mensagem clara a mudar o protocolo em silêncio. As principais verificações:

Onde	Verificação
script do modelo	Exige CUDA e `WORLD_SIZE=1` (uma GPU, para manter o batch efetivo 32).
`select_demonstrations`	`shots` precisa ser 0, 3 ou 5.
`make_messages`	Fine-tuned nunca recebe demonstrações.
`encode_prompt`	Prompt + tokens reservados para a resposta não podem passar de 16.384; não corta exemplos.
`make_demo_pool` / `indices_demonstracoes`	Pelo menos 5 demonstrações sem sobreposição com teste (e, no MedQuAD, com treino).
`textos_para_tokenizacao`	Textos de SFT precisam ser `str` não vazias; indica o índice inválido.
`answer_nll`	O prefixo tokenizado da conversa completa deve ser igual ao prompt; resposta precisa ter tokens.
`TokenTimer` / `timed_generate`	Batch 1, um token por callback, tokens do streamer iguais aos do `generate`, 160 tokens exatos no benchmark.
`rodar_todos.py`	Seeds no intervalo $[0, 2^{32})$, sem modelos ou seeds repetidos, `--repeticoes` ≥ 1.

TABELA 5: Verificações que interrompem a execução com `ValueError` ou `RuntimeError`.

4 ORQUESTRAÇÃO COM RODAR_TODOS.PY

4.1 OPÇÕES

Opção	Padrão	Efeito
`--modelos`	os 4	Quais modelos e em que ordem (`gemma4 huatuogpt medgemma qwen35`).
`--seeds`	`42`	Seeds de execução.
`--repeticoes`	`10`	Repetições por seed (use `1` para estudar seeds).
`--split-seed`	`42`	Seed fixa dos dados.
`--persona`	`clinico`	`clinico`, `biomedico` ou `original`.
`--saida`	`resultados_icl_personas`	Pasta da campanha.
`--planejar`	–	Só imprime o plano; não carrega modelos, dados nem GPU.
`--retomar`	–	Pula execuções completas com mesmo código e configuração.

TABELA 6: Argumentos de linha de comando.

4.2 ORDEM E ISOLAMENTO

O executor monta a lista de trabalhos com três laços: **modelo** → **seed** → **repetição**. Assim, um modelo termina todas as suas execuções antes de o próximo começar. Com `--seeds 42 123 2026 --repeticoes 1` são 12 processos e 12 fine-tunings.

Para cada trabalho, o executor cria `<saida>/<modelo>/run_NN/`, grava `status.json` com `status="rodando"` e chama o script com `subprocess.Popen`, usando a pasta como diretório de trabalho. A configuração vai por variáveis de ambiente:

```
LLM_RUN_DIR, LLM_SEED, LLM_SPLIT_SEED, LLM_PERSONA,
PYTHONUNBUFFERED=1, PYTHONIOENCODING=utf-8, MPLBACKEND=Agg
```

A saída do processo é mostrada no terminal e copiada linha a linha para `execucao.log`. O executor espera o processo terminar antes de abrir o próximo.

4.3 ESTADOS, FALHAS E RETOMADA

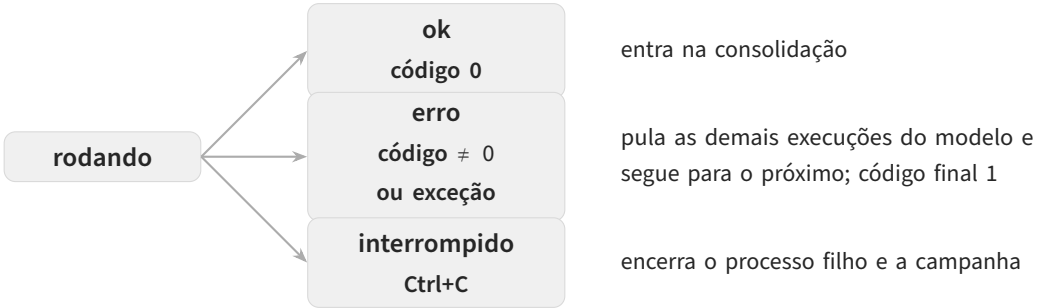


FIGURA 3: Estados gravados em `status.json` (com `duracao_s` e `exit_code`).

- **Falha**: o traceback vai para `execucao.log`, o processo filho é encerrado (`terminate`, depois `kill` após 20 s), resultados parciais ficam na pasta e são excluídos da consolidação.
- **Trava**: o arquivo `.em_execucao` impede dois executores na mesma campanha.
- **Manifesto**: `campanha.json` fixa protocolo, seeds, split, persona e repetições; se a configuração mudar, o executor pede outra `--saida`. Outro protocolo na mesma pasta também é recusado.
- **Retomada**: com `--retomar`, uma pasta existente só é pulada se `status="ok"` e todos os campos esperados batem, inclusive `codigo_sha256` (hash do script do modelo e de todos os `*.py` da raiz). Pastas com erro não são sobrescritas; não se retoma uma época no meio.

5 ARTEFATOS E LOGS

5.1 ESTRUTURA DE UMA CAMPANHA

```
campanha_clinico/
  campanha.json                configuracao fixa da campanha
  consolidado/                 saida de consolidar.py e resumir.py
  gemma4/run_01/
    status.json  execucao.log  ambiente.json  nvidia-smi.txt
    registros/                 prompts, dados, treino, tabelas
    figuras/                   PNGs dos graficos
    gemma4-e4b-medquad-unsloth/adapter-fine-tuned/
    gemma4-e4b-medquad-unsloth-extended/   MedQuAD por pergunta e resumo
    gemma4-e4b-medquad-unsloth-benchmark/  velocidade, MedQA, PubMedQA
  huatuogpt/run_01/ ...
```

5.2 REGISTRO.PY

`registro.py` só registra; não altera pesos, loss ou hiperparâmetros.

Função	O que faz
escrever_csv	Escreve em .tmp e renomeia (escrita atômica), UTF-8 com BOM, união das colunas de todas as linhas; listas e dicionários viram JSON.
iniciar	Cria pasta própria se o script rodar sozinho; grava ambiente.json (Python, plataforma, versões de torch/unsloth/transformers/trl/peft…, persona, prompts, seeds, hiperparâmetros) e nvidia-smi.txt.
etapa	Imprime [ETAPA n] descrição no log.
registrar_trainer	Callback que grava treino_<tag>_steps.csv (log_history) e config_<tag>.json (argumentos resolvidos do Trainer).
salvar_resultado	treino_<tag>_metricas.csv com training_loss, global_step e métricas finais.
salvar_tabelas	Linhas e DataFrames do script, conjuntos dados_*.csv, datasets.csv (tamanho e fingerprint) e model_config.json.
salvar_figuras	Salva todas as figuras abertas em figuras/ (130 dpi).

TABELA 7: Funções de registro.py.

Arquivo	Conteúdo
benchmark_summary.csv	Velocidade (média, mediana, P99/P1), acurácia com IC e métricas macro, por condição.
speed_per_question.csv	TTFT, TTLT, tempo total, tokens/s, prompt_tokens, token_times.
precision_per_question.csv	Letra prevista, resposta e acerto em MedQA/PubMedQA.
answers_per_question.csv	Resposta MedQuAD, referência, tempos, F1, ROUGE-L, NLL.
metrics_summary.csv	Agregado MedQuAD e métricas de treino por condição.
registros/prompts.csv	Mensagens completas, IDs das demonstrações, seed, persona, condição.
registros/previews.csv	Três respostas completas por condição.
registros/treino_fine-tuned_*	Loss, learning rate e métricas finais do SFT.

TABELA 8: Principais arquivos de resultado por execução.

5.3 PASTA AUDITORIA/

auditoria/ guarda o histórico das revisões do pacote: manifesto.json e manifesto_personas_v3.json (SHA-256 dos notebooks, scripts e documentos entregues), os diffs de cada script em relação ao original (gemma4.diff, personas_v3.diff, correcao_finetuning.diff...), PLANO_PERSONAS_V3.txt (saída de --planejar) e as notas VALIDACAO.md, REVISAO_INICIAL_HISTORICA.md e REVISAO_PERSONAS_V3.md. Esses arquivos descrevem versões anteriores; o código atual e o README.md prevalecem.

6 CONSOLIDAÇÃO E MÉTRICAS

6.1 MÉTRICAS POR EXECUÇÃO

MedQuAD (resposta aberta, até 160 tokens, greedy). Token F1 compara palavras normalizadas com multiplicidade; ROUGE-L usa a maior subsequência comum de palavras. Ambos medem semelhança lexical, não correção médica. A NLL soma a perda dos tokens da resposta de referência (dado o prompt da condição) e divide pelo total de tokens:

$$\text{NLL} = \frac{\sum_i \text{nll}_i}{\sum_i n_i}, \qquad \text{PPL} = e^{\text{NLL}}.$$

Também são registrados `finished_rate` (resposta terminou com EOS), número de palavras e pico de VRAM alocada pelo PyTorch.

Velocidade. O `TokenTimer` (um `BaseStreamer`) registra o instante de cada token gerado, com sincronização CUDA antes e depois do `generate`. Há um warm-up de 8 tokens fora da medição.

$$\text{TTFT} = t_1 - t_0, \qquad \text{TTLT} = t_N - t_0, \qquad \text{tokens/s} = \frac{N-1}{\text{TTLT} - \text{TTFT}}.$$

`benchmark_summary.csv` traz média, mediana e a cauda do pior caso: P99 para tempos (menor é melhor) e P1 para tokens/s (maior é melhor). Com $n = 20$ o P99 é quase o máximo, e o código emite um aviso (use $n \geq 100$).

MedQA e PubMedQA. `pick_option` acrescenta `"Answer:"` ao prompt e escolhe a letra com o maior logit do próximo token (considerando “A” e “ A”). A acurácia tem IC aproximado $\bar{x} \pm 1{,}96\, s/\sqrt{n}$, limitado a $[0, 1]$. Precision, recall e F1 macro são calculados por alternativa (A–D ou A–C) e depois tirada a média simples; não são F1 por especialidade.

6.2 CONSOLIDAR.PY

- Lê todos os `*/run_*/status.json`; se houver mais de um protocolo na pasta, para com erro.
- Grava `consolidado/execucoes.csv` com todas as execuções, inclusive as que falharam.
- Para execuções com `status="ok"`, junta cada CSV pelo nome do arquivo, ignora `dados_*` e acrescenta `exec_modelo`, `exec_repeticao`, `exec_seed`, `exec_split_seed`, `exec_persona` e `exec_origem`.
- CSVs da pasta `registros/` recebem o prefixo `registro__`; são cópias e não devem ser somadas aos CSVs originais.

6.3 RESUMIR.PY

Agrega entre execuções. Para cada grupo calcula n, média, desvio-padrão, mínimo, máximo e IC 95% da média com t de Student:

$$\bar{x} \pm t_{0{,}975,\, n-1}\, \frac{s}{\sqrt{n}}.$$

Saída em consolidado/	Conteúdo
resumo_entre_execucoes.csv	Resumo de benchmark_summary por modelo, categoria, métrica e condição.
deltas_pareados.csv	Diferença de cada condição contra o zero-shot da mesma execução.
resumo_deltas.csv	Resumo desses deltas (3-shot, 5-shot e fine-tuned − zero-shot).
resumo_medquad_condicoes.csv	Resumo de metrics_summary por modelo, regime e métrica.

TABELA 9: Arquivos gerados por resumir.py.

O IC entre execuções descreve a variação observada entre execuções; não substitui a incerteza entre perguntas.

7 COMO REPRODUZIR

Ambiente. É preciso uma GPU NVIDIA com CUDA e Python com PyTorch, Unsloth, Transformers e TRL compatíveis com os checkpoints. requirements.txt lista só as dependências auxiliares (tqdm, pandas, numpy, matplotlib, ipython, datasets, sentencepiece, huggingface_hub, scipy); a pilha de treino segue a instalação oficial do Unsloth. Checkpoints restritos exigem login no Hugging Face.

```
# 1. Dependencias auxiliares (depois de PyTorch CUDA + Unsloth)
python -m pip install -r requirements.txt

# 2. Conferir o plano (sem GPU, sem downloads)
python rodar_todos.py --repeticoes 1 --seeds 42 123 2026 --persona
   clinico --planejar

# 3. Piloto com um modelo
python rodar_todos.py --modelos medgemma --repeticoes 1 --seeds 42 --
   saida piloto
python -m pip freeze > ambiente_validado.txt

# 4. Campanha completa: 4 modelos x 3 seeds = 12 processos
python rodar_todos.py --repeticoes 1 --seeds 42 123 2026 --persona
   clinico --saida campanha_clinico

# 5. Retomar apos interrupcao (pula so execucoes completas)
python rodar_todos.py --repeticoes 1 --seeds 42 123 2026 --persona
   clinico --saida campanha_clinico --retomar

# 6. Consolidar e resumir (a consolidacao tambem roda apos cada processo)
python consolidar.py campanha_clinico
python resumir.py campanha_clinico
```

Boas práticas

- Não atualize bibliotecas durante uma campanha; use a mesma GPU para todos os modelos.
- Mudou código, persona, seeds ou repetições: use uma nova `--saida`.
- Para estudar seeds, prefira `--repeticoes 1`; repetições da mesma seed não são amostras independentes.
- Outra persona é outra campanha: `--persona biomedico --saida campanha_biomedico`.

8 LIMITAÇÕES E PONTOS DE ATENÇÃO

- **Sem execução real nesta documentação**: o texto foi escrito a partir do código; não houve download de modelos, inferência ou treino em GPU.
- **Testes ausentes**: o `README.md` cita `python -m unittest discover -s testes -v`, mas a pasta `testes/` foi removida do repositório; o comando não funciona na versão atual.
- **Nome do protocolo**: `status.json` e `campanha.json` usam `icl_0_3_5_ft5_personas_v4_ft_separado` (`rodar_todos.py`), mas `ambiente.json` grava `icl_0_3_5_ft5_personas_v3` (`prompts_medicos.py`). Não afeta a execução; vale alinhar para evitar confusão na auditoria.
- **Retomada**: `codigo_sha256` cobre os `*.py`, mas não `prompts_prontos.json` (que é só referência; os prompts usados vêm de `prompts_medicos.py`).
- **HuatuoGPT**: avaliação principal e velocidade usam 160 tokens, o que pode cortar o raciocínio antes de `## Final Response`.
- **Métricas**: F1 e ROUGE-L são lexicais; perplexidade depende do tokenizador e não serve como ranking direto entre famílias; a escolha de letra pelo próximo token supõe letras de um único token.
- **Comparação ICL × FT**: em MedQA e PubMedQA o few-shot usa exemplos do próprio benchmark, enquanto o fine-tuning usa apenas MedQuAD.